

Workshop Registration Service

Local setup instructions — running the frontend and the backend on your machine

Stack: NestJS 12 + Prisma 6 + PostgreSQL (API) | Next.js 16 (React 19) + plain CSS (UI) | **Verified on:** macOS, Node v22.17.0, npm 11.5.2

Everything below is copy-paste. Run the steps in order and you will have the API on `http://localhost:3001`, the UI on `http://localhost:3000`, three working logins and four sample workshops — no hosted database required.

1. What you need

Requirement	Version used	Notes
Node.js + npm	Node 20+ (v22.17.0)	Both apps are Node/TypeScript. Check with <code>node -v</code> .
PostgreSQL	14 or newer	Required: the Prisma schema declares <code>provider = "postgresql"</code> , and the capacity rule depends on Postgres row locking (<code>SELECT ... FOR UPDATE</code>). SQLite cannot express it.
Git	any	To clone the repository.
Docker (optional)	—	Only the quickest way to get a disposable Postgres (option B below); a normal local install is fine too.

Default ports (free them, or override — see the tables in each step):

Service	Port	How to change it
Backend API	3001	<code>PORT=3002 npm run start:dev</code>
Frontend	3000	<code>npm run dev -- -p 3005</code>
PostgreSQL	5432	Wherever you installed it (5433 in the Docker example)

2. Repository layout

```
workshop_registration/
├── backend/                NestJS API
│   ├── prisma/schema.prisma  models: User, Workshop, Registration, AuditLog
│   ├── prisma/seed.ts        demo logins + sample workshops
│   ├── prisma/migrations/    committed SQL migrations
│   └── src/                  auth/ users/ workshops/ registrations/
├── frontend/               Next.js UI (overview, workshops, users, activity)
├── README.md               status, deployment notes, issues & solutions
└── docs/                   this document + the design/trade-offs document
```

3. Step 1 — get the code

```
git clone <repository-url> workshop_registration
cd workshop_registration
```

Two separate `package.json` files — one per app — so install and run each folder on its own. Use two terminals (or two terminal tabs): one for the API, one for the UI.

4. Step 2 — a PostgreSQL database

Pick the line that matches your machine. All three produce the same `DATABASE_URL` shape.

Option A — PostgreSQL already installed (Homebrew on macOS)

```
brew services start postgresql@16
createdb workshop_registration
# DATABASE_URL: postgresql://<your-macos-username>@localhost:5432/workshop_registration
```

Homebrew's Postgres trusts local connections and sets no password by default, so the username is simply whoever you are (whoami). On Linux use `sudo -u postgres createdb workshop_registration`.

Option B — Docker (nothing to install, disposable)

```
docker run --name wr-postgres -e POSTGRES_PASSWORD=postgres -p 5433:5432 -d postgres:16
docker exec wr-postgres createdb -U postgres workshop_registration
# DATABASE_URL: postgresql://postgres:postgres@localhost:5433/workshop_registration
# Stop it with: docker stop wr-postgres (start again: docker start wr-postgres)
```

Option C — a free Neon Postgres (same engine as the live deployment)

Create a project at neon.com, click **Connect** and copy *both* connection strings: the pooled one goes into `DATABASE_URL`, the direct (unpooled) one into `DIRECT_URL`, because migrations cannot run through the pooler. Keep the `?sslmode=require` the dashboard adds and append `&connect_timeout=15` — Neon suspends idle computes and without it the first query can fail with "Can't reach database server".

Only the database name is created here. The tables come from the migrations in step 4 — you do not need to write any SQL by hand.

5. Step 3 — install the backend and point it at the database

```
cd backend
npm install          # NestJS, Prisma, bcrypt, vitest, oxlint
npx prisma generate  # generates the typed Prisma client into node_modules
```

Create `backend/.env` — the file is git-ignored, so real credentials never reach the repository:

```
DATABASE_URL="postgresql://postgres@localhost:5432/workshop_registration"
DIRECT_URL="postgresql://postgres@localhost:5432/workshop_registration"
JWT_SECRET="change-me-to-a-long-random-string"
PORT=3001
# CORS_ORIGIN="http://localhost:3000,https://your-site.netlify.app" # only to lock CORS down
```

Variable	Required?	What it does
DATABASE_URL	Yes	Runtime connection string used by Prisma Client.
DIRECT_URL	Yes*	Unpooled connection used by migrations. Locally set it to the same value as DATABASE_URL.
JWT_SECRET	Recommended	Signs and verifies the login tokens. Without it the API falls back to a development-only constant — acceptable locally, but a deployment must set it, otherwise anyone holding the source can forge an ADMIN token.
PORT	No	Defaults to 3001.
CORS_ORIGIN	No	Comma-separated allow-list. Empty means CORS is open, which is what you want for local work and for any *.netlify.app preview URL.

Signed in yesterday and now every request is 401? `JWT_SECRET` changed (or was added), so old tokens no longer verify. Sign out and sign in again — nothing is broken.

6. Step 4 — create the tables and load the demo data

```
npx prisma migrate deploy  # applies the committed migrations (creates all tables)
npm run db:seed            # 3 logins + 4 sample workshops
```

The seed prints what it created and is safe to run repeatedly: users are upserted without resetting an existing password, and registrations are only inserted into workshops that have none, so your own test data is never wiped.

Seeded development data:

ADMIN admin@test.com password: admin123

MANAGER manager@test.com password: manager123

STAFF staff@test.com password: staff123

4 workshops: POT-101, COD-201, FIT-301, CER-110 – FIT-301 is full with 1 waitlisted

Dev-only credentials – replace them before any real deployment.

Code	Workshop	Capacity	Why it is in the seed
POT-101	Pottery Basics	12 (2 taken)	Seats left — a normal registration flow.
COD-201	Intro to Coding	20 (1 cancelled)	Shows a cancellation in the history, with who and when.
FIT-301	Morning Fitness	4 (full)	Full with one attendee already waitlisted — register again and you join the queue; cancel an active seat and the queue is promoted.
CER-110	Ceramics Glazing	8	Past date and COMPLETED status, for the filters.

Prefer a GUI? `npx prisma studio` opens a table browser for your database at `http://localhost:5555`. To start over from an empty database: `npx prisma migrate reset` (drops everything, re-runs migrations, re-seeds automatically).

7. Step 5 — run the backend API

```
cd backend
npm run start:dev      # watch mode, restarts on save -> http://localhost:3001
```

For a production-style run instead: `npm run build && npm run start:prod`.

Confirm it is up before moving on:

```
curl http://localhost:3001/health
# {"status":"ok"}
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:3001/users
# 401 - the request had no token, which is the correct answer
```

The API surface, and which role each route accepts:

Endpoint	Allowed roles	Purpose
POST /auth/login	public	Returns the JWT, the role and the email.
GET /health	public	Liveness probe for hosting platforms.
POST /users	ADMIN	Create an account and set its role. There is no public signup.
GET /users	ADMIN	List accounts.
GET /users/audit	ADMIN, MANAGER, STAFF	Audit trail (who did what, when).
GET /workshops	MANAGER, STAFF	Catalogue plus the active/seat counts used by the filters.
GET /workshops/:id	MANAGER, STAFF	One workshop with its full registration history.
POST /workshops	MANAGER	Add a workshop (409 if the code is already used).
PATCH /workshops/:id	MANAGER	Edit a workshop (title, instructor, date, capacity, status).
POST /registrations	MANAGER, STAFF	Register an attendee — ACTIVE, or WAITLISTED when the workshop is full.
PATCH /registrations/:id/cancel	MANAGER, STAFF	Cancel, keep the row, and promote the next waitlisted attendee.

An ADMIN token cannot list workshops (403) — by design, from the roles table in the brief. Browse the catalogue as `manager@test.com` or `staff@test.com`.

8. Step 6 — run the frontend

```
cd ../frontend      # from the repository root: cd frontend
npm install
npm run dev         # -> http://localhost:3000
```

Create `frontend/.env.local` to tell the UI where the API lives:

```
NEXT_PUBLIC_API_URL=http://localhost:3001
```

Situation	What to do
Local development	<code>.env.local</code> is optional — the code already falls back to <code>http://localhost:3001</code> .
Local production build	Set it if the API is on another port or host, then <code>npm run build</code> .
Netlify	Mandatory. The value is baked into the bundle at build time, so a build without it produces a site that only talks to your own laptop. <code>next.config.ts</code> therefore fails the Netlify build with a clear message when the variable is missing or still points at localhost.

9. Step 7 — sign in and try it out

Open `http://localhost:3000`, sign in with one of the seeded accounts, and use the sidebar to switch views. The email is the only thing that changes what you can see.

Email	Password	Role	What you can do
<code>admin@test.com</code>	<code>admin123</code>	ADMIN	Create accounts and assign roles; read the audit trail.
<code>manager@test.com</code>	<code>manager123</code>	MANAGER	Add and edit workshops, register and cancel attendees, see history.
<code>staff@test.com</code>	<code>staff123</code>	STAFF	Register and cancel attendees, see history.

Five-minute click-through that exercises every requirement:

1. **Sign in as staff** — the workshops list loads with seat counters and no user-management menu.
2. **Overview** — type a date range, or filter by status, or tick "seats available" only. That is requirement 4.
3. **Workshops** → **FIT-301 (Morning Fitness)** — it shows 4/4 seats. Register an attendee: the response is a **waitlist** row, not an error, and the seat count does not move.
4. **Cancel** one of the active registrations on that same workshop. The cancelled row stays in the history with your email and the time, and the person from step 3 is promoted to **ACTIVE** in the same instant the seat frees.
5. **Sign in as manager** — add a workshop, then edit its capacity. Try the same code twice to see the 409 message.
6. **Sign in as admin** — create a user with any role, then open **Activity** to see the trail: account creation, workshop edits and cancellations, each with the actor and a timestamp.

10. Everyday commands

Command	What it does
<code>cd backend && npm run start:dev</code>	API with hot reload (port 3001).
<code>cd frontend && npm run dev</code>	UI with hot reload (port 3000).
<code>cd backend && npm test</code>	Unit tests (Vitest) — currently 6 passing, including the capacity and waitlist rules.
<code>cd backend && npm run lint</code>	oxlint over <code>src/</code> and <code>test/</code> .
<code>cd backend && npm run build</code>	Type-check and compile to <code>dist/</code> .
<code>cd backend && npm run db:seed</code>	Re-run the seed at any time; it never overwrites your data.
<code>cd backend && npx prisma studio</code>	Browse and edit the database in the browser.
<code>cd backend && npx prisma migrate reset</code>	Drop everything, re-migrate and re-seed — a clean slate.
<code>cd frontend && npm run build</code>	Static export into <code>frontend/out</code> (this is what Netlify publishes).

11. If something does not work

Symptom	Cause and fix
The login page says <i>"Unable to reach the server"</i>	The API is not running, or the UI is pointed elsewhere. Check <code>curl http://localhost:3001/health</code> ; if that works, fix <code>NEXT_PUBLIC_API_URL</code> in <code>frontend/.env.local</code> and restart <code>npm run dev</code> (Next.js inlines the value at build/start time).
Can't reach database server / Prisma P1001	Postgres is not running or the URL is wrong. <code>brew services list</code> , <code>docker ps</code> , or try <code>psql "\$DATABASE_URL"</code> . For Neon, add <code>&connect_timeout=15</code> — the first connection after the compute wakes can take a few seconds.
Environment variable not found: <code>DATABASE_URL</code>	<code>backend/.env</code> is missing or in the wrong folder. It must sit next to <code>backend/package.json</code> .
TypeScript cannot find <code>@prisma/client</code>	Run <code>npx prisma generate</code> (step 3). It must be repeated after changing <code>schema.prisma</code> .
EADDRINUSE on 3000 or 3001	Something already owns the port. <code>lsof -ti:3001 xargs kill</code> , or use another one: <code>PORT=3002 npm run start:dev</code> and the matching <code>NEXT_PUBLIC_API_URL</code> .
401 Invalid signature right after signing in	Two servers with different <code>JWT_SECRET</code> values, or the secret changed under a live token. Restart the API with one secret and sign in again.
400 with "property X should not exist"	The API rejects unknown body fields on purpose. Send only the fields the endpoint documents.
Registration answered WAITLISTED instead of ACTIVE	Not a bug: the workshop is full. Cancel an active registration and the first person in the queue (oldest first) is promoted automatically.
Nothing appears in the Activity view	Expected on a brand-new database — the list fills as soon as you create a user, edit a workshop, register or cancel. <code>npm run db:seed</code> also writes a few entries.

One-repo layout: the backend and the frontend live side by side, each with its own `package.json`. There is no monorepo tooling to install — two terminals and the steps above are the whole story. Deployment notes (Render + Netlify + Neon) are in `README.md`; the reasoning behind the technology choices, the design decisions and the capacity rule is in *Technology Choices, Design Decisions and Trade-offs*.